

Artifact 存储与工作区管理

SPEC-005 | Status: 设计中 | 依赖: SPEC-003 (SeaweedFS 连接层), SPEC-004 (Session Manager)

目标

建立 Artifact 数据模型、SeaweedFS 存储接口、工作区 (Workspace) 文件管理 API, 为 `save_artifact`、`workspace_read/write/exec` 等 Skill 提供底层基础设施。

背景

Roadmap Phase 2 (Week 3), P2-08 (Artifact 存储), P2-09 (Session Manager 工作区)。Artifact 是所有分析产物的存储基础——报告、图表、数据导出等最终都以 Artifact 形式持久化。工作区则是每个 Session 的临时文件隔离空间。

前置依赖检查

前置 Spec	状态	备注
SPEC-002	✅/❌	CI Pipeline 就绪
SPEC-003	✅/❌	SeaweedFS 连接层可用
SPEC-004	✅/❌	Session Manager + SkillContext 可用

详细设计

1. Artifact 数据模型

```
// Artifact 元数据 - MongoDB artifacts collection
type Artifact struct {
    ID          string    `bson:"_id"`
```

```

UserID      string    `bson:"user_id"`
SessionID   string    `bson:"session_id"`
TaskID      string    `bson:"task_id,omitempty"` // Agent 任务关联
Name        string    `bson:"name"`             // 文件名
MimeType    string    `bson:"mime_type"`
SizeBytes   int64     `bson:"size_bytes"`
StoragePath string    `bson:"storage_path"` // SeaweedFS 路径
Persistent  bool       `bson:"persistent"` // false=临时, true=永久
CreatedAt   time.Time `bson:"created_at"`
UpdatedAt   time.Time `bson:"updated_at"`
}

```

- `Persistent=false`: Session 结束后 TTL 自动清理
- `Persistent=true`: 永久保存, 如分析报告、导出的图表
- 通过 `artifact_id` 单向引用, 报告/知识库等上层表不反向侵入 artifacts

2. SeaweedFS 存储层

```

type ArtifactStorage struct {
    seaweedfs *SeaweedFSClient
}

func (s *ArtifactStorage) Upload(userID, sessionID string, name string, mimeType string,
reader io.Reader, persistent bool) (*Artifact, error) {
    path := fmt.Sprintf("%s/%s/%s", userID, sessionID, name)
    size, err := s.seaweedfs.Upload(path, reader)
    // 写入 MongoDB 元数据
    artifact := &Artifact{ID: uuid.New().String(), ...}
    s.mongo.Insert("artifacts", artifact)
    return artifact, nil
}

func (s *ArtifactStorage) Download(artifactID string) (io.ReadCloser, error) {
    artifact := s.mongo.FindOne("artifacts", bson.M{"_id": artifactID})
    return s.seaweedfs.Download(artifact.StoragePath)
}

func (s *ArtifactStorage) Delete(artifactID string) error {
    artifact := s.mongo.FindOne(...)
}

```

```
s.seaweedfs.Delete(artifact.StoragePath)

return s.mongo.Delete("artifacts", artifactID) // 幂等

}
```

3. 工作区 (Workspace) 管理

```
type WorkspaceManager struct {
    storage      *ArtifactStorage
    sessionMgr   *SessionManager
    basePath     string // "workspace/{user_id}/{session_id}"
}

// ReadFile 读取工作区文件内容 (按 session 隔离)
func (w *WorkspaceManager) ReadFile(sessionID, filename string) ([]byte, error) {
    path := fmt.Sprintf("%s/%s/%s", w.basePath, sessionID, filename)
    return w.storage.Read(path)
}

// WriteFile 写入文件到工作区
func (w *WorkspaceManager) WriteFile(sessionID, filename string, data []byte) error {
    path := fmt.Sprintf("%s/%s/%s", w.basePath, sessionID, filename)
    return w.storage.Write(path, data)
}

// ExecScript 在沙箱中执行脚本 (仅限工作区路径)
func (w *WorkspaceManager) ExecScript(sessionID, script string) (string, error) {
    // 沙箱执行: 限制文件系统访问范围、CPU/内存、超时
    // 仅允许操作 basePath/{sessionID}/ 下的文件
    return sandbox.Exec(script, sandbox.WithRoot(w.sessionPath(sessionID)))
}

// ListWorkspace 列出工作区文件
func (w *WorkspaceManager) List(sessionID string) ([]FileInfo, error) { ... }

// CleanupWorkspace 清理过期 session 的工作区文件
func (w *WorkspaceManager) Cleanup(sessionID string) error { ... }
```

4. Session 工作区隔离

- 每个 Session 拥有独立的工作区路径: `workspace/{user_id}/{session_id}/`
- Session A 无法访问 Session B 的文件
- Session 过期后自动清理工作区文件 (MongoDB TTL + SeaweedFS 级联)
- 临时 Artifact 关联 session_id, Session 结束时标记清理

5. Artifact API

Method	Path	说明
POST	<code>/api/v1/artifacts/upload</code>	上传文件 (multipart)
GET	<code>/api/v1/artifacts/{id}/download</code>	下载文件 (流式)
DELETE	<code>/api/v1/artifacts/{id}</code>	删除文件 (幂等)
GET	<code>/api/v1/artifacts?session_id=x</code>	列出 Session 文件
GET	<code>/api/v1/workspace/{session_id}/files</code>	列出工作区文件

6. 批量下载 (F-21)

- `GET /api/v1/tasks/{task_id}/artifacts/download` 一 打包下载任务全部 Artifact
- 后端 zip 打包 + SeaweedFS 流式读取 → HTTP 流式返回
- 不占内存, 适合大文件

可行性分析

检查项	结论
是否需要新 DB 集合	Yes (artifacts)
是否影响现有 API	No
性能影响	SeaweedFS 上传/下载 ≤ 2s (<10MB 文件)
是否需要新增 Skill	No (本 spec 是 Skill 的底层基础设施)
是否需要 E2E 测试	Artifact 上传→下载→删除 UI 用例

相关文件

文件	角色	变更幅度
<code>internal/domain/artifact/</code>	Artifact 领域模型	新建
<code>internal/infra/seaweedfs/</code>	SeaweedFS 存储客户端	新建
<code>internal/service/artifact/</code>	Artifact 业务逻辑 + API	新建
<code>internal/logic/workspace/</code>	工作区管理	新建

验证标准

1. 上传文件 → SeaweedFS 落盘 → MongoDB artifacts 记录 → 返回 artifact_id
2. 下载文件 → 流式返回，不占内存
3. 删除文件 → SeaweedFS + MongoDB 级联删除，幂等（不存在的文件不报 404）
4. Session A 的工作区文件 Session B 不可见
5. 过期 Session 的工作区文件自动清理